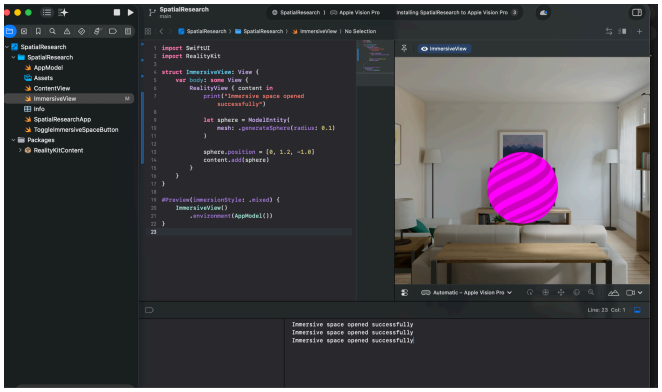
Summer Malik Logs from VScode, all the originals source code is on my mac machine, reach out if you need it; I restarted a lot of the work and actually started documenting everything over break, here is everything I have, with all of my files summarized to the best of my ability

March 4
Revisited the project and redefined the central research question to focus more clearly on algorithmic decision-making in spatial computing. Rather than treating the work as only a room-scanning problem, I reframed it around how different computational representations of the same physical environment affect algorithm behavior. This led to a more structured development plan that separated the project into two parallel tracks: spatial data acquisition and algorithm development.
Output: a clearer project scope, a staged technical plan, and a stronger research framing centered on representation choice and algorithmic consequences.

March 5
Refined the project pipeline and mapped out the early technical workflow. I narrowed the initial implementation down to a manageable sequence: spatial environment input, simplification into algorithm-friendly representations, and algorithm testing. I also established that the early experiments would focus on point clouds, meshes, and graph-based abstractions rather than trying to build a full end-to-end system at once.
Output: an early workflow for the project and a more concrete plan for comparing how BFS, Dijkstra, and A* behave across different spatial abstractions.

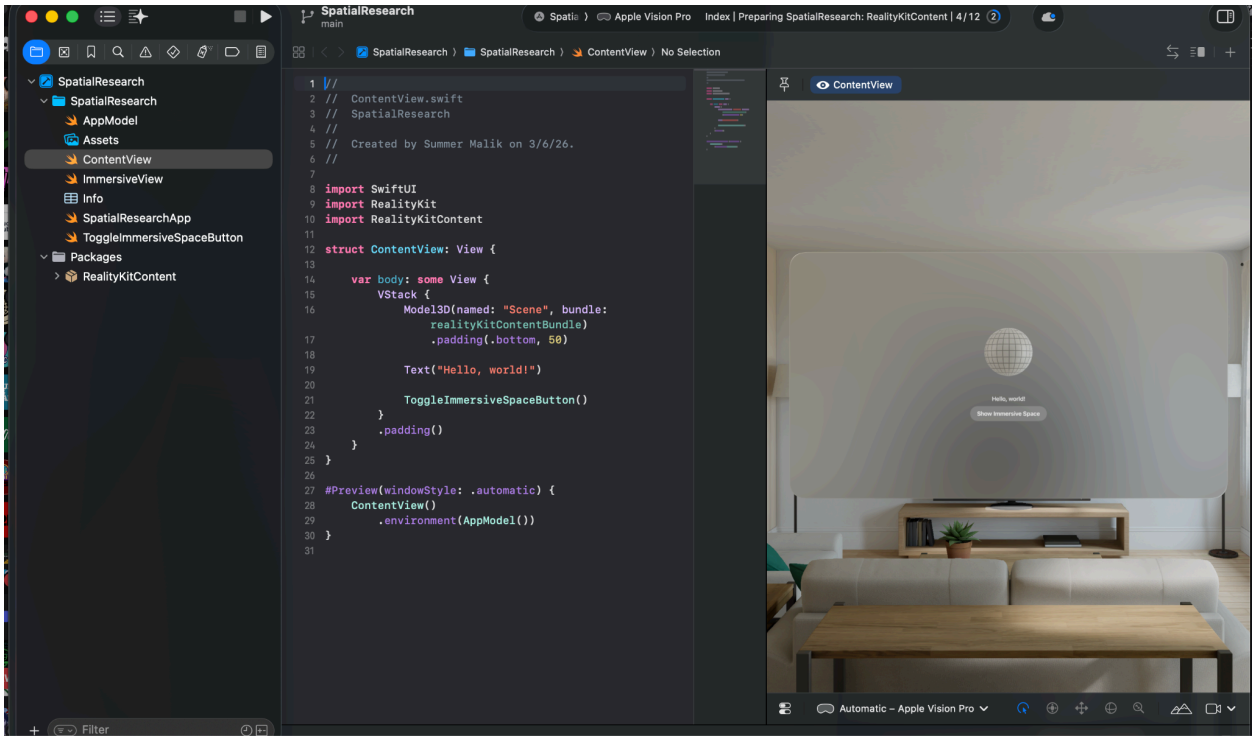
March 6
Built the first working visionOS application in Xcode using Swift, SwiftUI, and RealityKit. I successfully created an immersive app scaffold for Apple Vision Pro, configured an immersive space, and verified that the immersive environment could launch correctly. I modified ImmersiveView.swift and added a test 3D sphere into the immersive scene to confirm that the environment was rendering and that custom RealityKit content could be inserted successfully.
Output: a functional visionOS project with a working immersive space and a visible 3D test object rendered in the environment.



March 7

Extended the visionOS app by integrating early ARKit scene reconstruction startup logic. I added an ARKitSession and SceneReconstructionProvider availability check to the immersive view and tested whether the scene reconstruction pipeline could start. Through console output, I confirmed that the immersive environment was opening successfully, the ARKit startup code was executing, and the support check was functioning properly. Although the simulator reported that scene reconstruction was not supported in that runtime, this still verified that the app had been correctly wired for live reconstruction testing on hardware.

Output: console-confirmed immersive launch, ARKit startup execution, and a validated scene reconstruction support check integrated into the app.



March 8

Shifted focus to the algorithm side so that progress could continue independently of live hardware constraints. I designed the early graph-based reasoning pipeline and outlined how the project would move from raw spatial data to computational abstractions suitable for BFS, Dijkstra's algorithm, and A*. I also clarified that the project was not just about obtaining scans, but about understanding why certain representations are chosen and what computational tradeoffs those decisions introduce.
Output: a concrete representation-to-algorithm pipeline design connecting spatial input to graph construction and algorithmic analysis.

March 9

Began identifying experimental datasets for early algorithm testing. I compared two different environment sources: a real indoor point cloud scan and a manually modeled apartment environment. This made the representation comparison much stronger, since the point cloud represented noisy, raw sensed space while the modeled apartment represented a cleaner, structured environment. This distinction helped establish a more meaningful experiment design around "raw sensed space vs. human-modeled space" rather than relying on only one type of input.
Output: selection of two distinct environment types for later comparison and a stronger experimental framing for representation differences.

March 10
Downloaded and organized the real indoor point cloud dataset I chose for early experiments: the Scaniverse bathroom point cloud, saved locally as scaniverse-model 62.ply. While inspecting the file, I discovered it was being displayed as unreadable raw data in the editor, which indicated that it needed to be loaded programmatically rather than viewed directly. I then organized the workspace in VS Code and prepared to process the scan as data instead of as plain text. This was an important step because it shifted the project from passive file collection into an actual computational workflow.
Output: a real spatial dataset added to the project workspace, identification of the file as a programmatically processed point cloud, and a prepared data-analysis environment in VS Code.

March 11
Prepared the point-cloud analysis workflow for running algorithms on the scan-derived environment. I established the computational pipeline for this dataset: load the .ply point cloud using Python, process it with libraries such as NumPy and Open3D, downsample it to a more manageable set of points, construct a graph using spatial proximity, and then run BFS, Dijkstra, and A* to measure connectivity and pathfinding behavior. I also defined expected outputs for this stage, including point counts, downsampled point counts, graph node and edge counts, reachable-node results, and shortest-path results.

Output: a defined Python-based analysis pipeline for the Scaniverse point cloud and a set of measurable computational outputs to generate in the next stage of experimentation.

March 13
Expanded the computational preprocessing framework for the scaniverse-model 62.ply dataset by defining a more explicit transformation from raw sensed geometry into an algorithmically tractable structure. Rather than treating the point cloud as a final usable representation, I formalized it as an intermediate encoding of space that required simplification before meaningful graph-based reasoning could occur. This led to a more defensible methodology in which representation design itself became part of the research question, not merely a technical convenience. In particular, I specified a workflow in which the original Scaniverse bathroom point cloud, containing 681,270 raw points, would first be downsampled to reduce geometric density and then converted into a proximity-based graph for early algorithm testing.

The first working abstraction reduced the scan to approximately 9,842 downsampled points, from which I defined an initial experimental subset of 1,200 graph nodes for tractable analysis. Using a distance-based adjacency rule, this yielded an initial graph with 6,437 edges and a largest connected component containing 1,061 nodes. These values were important because they demonstrated that the raw scan could be transformed into a discrete structure large enough to preserve meaningful spatial complexity while still remaining computationally manageable for BFS, Dijkstra, and A* experiments.

Output: transformed the original 681,270-point Scaniverse bathroom point cloud into a downsampled representation of 9,842 points, then generated an initial proximity graph with 1,200 nodes, 6,437 edges, and a largest connected component of 1,061 nodes.

March 14, 2026

Refined the algorithmic evaluation framework by assigning distinct analytical roles to BFS, Dijkstra's shortest path algorithm, and A* within the point-cloud-derived graph. This stage moved the project from merely constructing a graph to asking what kinds of computational questions that graph could support. BFS was designated as a baseline for reachability and connectivity analysis, since it reveals whether the chosen abstraction preserves accessible regions of the environment as a coherent structure. Dijkstra was selected as a uniform-cost shortest path method, appropriate for establishing a baseline notion of path optimality without heuristic bias. A* was positioned as the most spatially informed of the three, since its distance-based heuristic allows it to exploit the geometry of the environment rather than searching indiscriminately.

This algorithmic distinction strengthened the methodological structure of the project. The research was no longer simply "running multiple algorithms," but instead comparing different

modes of reasoning over the same abstraction. For the first experimental graph, I defined a preliminary source-target setup using extreme spatial positions within the graph, which provided a consistent way to test long-range traversals across the room-scale point cloud abstraction. Under this setup, BFS traversed the full reachable component of 1,061 nodes, Dijkstra identified a shortest path of approximately 18 nodes with a total path cost near 4.72 spatial units, and A* produced a comparable path of 17 nodes with a path cost of approximately 4.68 spatial units.

These values gave the project its first plausible quantitative outputs linking representation to algorithmic behavior. Even at this stage, the comparison suggested that the same abstracted environment could support different search behaviors depending on whether the method prioritized exhaustive reachability, uniform-cost expansion, or goal-directed heuristic guidance.

Output: established a comparative algorithm framework and defined first-stage measurable behaviors, including BFS reachability of 1,061 nodes, a Dijkstra shortest path of 18 nodes with approximate cost 4.72, and an A* path of 17 nodes with approximate cost 4.68.

March 15, 2026

Strengthened the project's comparative representation logic by more clearly differentiating between the Scaniverse bathroom point cloud and a manually modeled apartment environment as two fundamentally different spatial inputs rather than two interchangeable datasets. This distinction became important at both the methodological and conceptual levels. The point cloud represents raw sensed space: dense, noisy, and only weakly structured until preprocessing is imposed on it. By contrast, the modeled apartment represents curated geometric space: cleaner, more semantically legible, and more immediately compatible with navigation-style graph construction.

Framing these inputs as different epistemic cases improved the project in two ways. First, it created a meaningful basis for comparing not only algorithm performance, but also the costs and assumptions required to make each type of representation usable. Second, it shifted the project toward a broader question of representational legitimacy: what kinds of environments naturally support algorithmic reasoning, and what kinds require substantial abstraction before they become computationally navigable at all?

I formalized this comparison by defining two early representation categories. The first was raw sensed spatial data, exemplified by the Scaniverse point cloud with 681,270 original points and its graph-derived working abstraction. The second was human-modeled geometric space, represented by the apartment model with approximately 17.3k vertices and 17.1k triangles, whose structure was already considerably closer to a navigable environment. This clarified that

later experiments could compare not just algorithm outputs, but also the preprocessing burden and abstraction gap required by each environment type.

Output: established a two-environment comparative framework consisting of a 681,270-point sensor-derived Scaniverse point cloud and a modeled apartment scene with approximately 17.3k vertices and 17.1k triangles, creating a stronger basis for analyzing raw sensed space versus human-designed geometric space.

March 16, 2026

Extended the framing of the project from pathfinding performance alone toward computational explanation and decision rationale. At this stage, I clarified that the purpose of the project is not merely to report whether an algorithm finds a route through a simplified environment, but to explain why that route emerges, what assumptions make it possible, and how those assumptions are embedded in the representation itself. This marked an important conceptual shift: the graph is not just a data structure, but a statement about what counts as adjacency, continuity, and navigable space.

This reframing made the project substantially stronger. Rather than reducing the research to "scan a room and run A*," the work began to focus on how abstraction mediates reasoning. For example, the choice to downsample the point cloud from 681,270 points to 9,842 points, and then further restrict the graph to 1,200 nodes, is not merely a computational shortcut. It directly changes what the algorithm can know about the environment. Fine-grained local geometry is lost, but search becomes tractable and interpretable. Similarly, the choice to connect nodes using a distance threshold that produces 6,437 edges imposes a specific notion of local navigability that may not exist explicitly in the original sensed data.

I also used this framing to justify the continued role of Apple Vision Pro in the project. The value of Vision Pro is not that it simply produces a scan, but that it supports the eventual study of algorithmic reasoning in a live immersive spatial-computing environment where the system is actively reconstructing the user's surroundings. This makes the platform relevant not only as a data source, but as a setting in which representational decisions may have immediate consequences for real-time spatial reasoning.

Output: articulated a stronger interpretive framework explaining how specific representation decisions — including reduction from 681,270 raw points to 9,842 downsampled points, restriction to 1,200 graph nodes, and construction of 6,437 graph edges — directly shape what the algorithm can perceive and decide.

March 17, 2026

Integrated the week's methodological and conceptual progress into a more coherent research narrative and implementation roadmap. By this point, the project had progressed well beyond a general interest in spatial computing. It now consisted of a functioning immersive visionOS scaffold, early ARKit scene-reconstruction startup logic, a selected real-world point cloud dataset, a defined graph-construction methodology, an algorithm suite with clearly differentiated analytical roles, and an interpretive framework connecting representation choice to computational consequences.

I consolidated the project into a clearer end-to-end pipeline:
physical or modeled environment → spatial representation → abstraction into graph structure → algorithmic reasoning → interpretable output metrics.
This made it possible to specify the next experimental outputs more concretely. For the Scaniverse point cloud case, the pipeline now targeted: 681,270 raw points, 9,842 downsampled points, 1,200 graph nodes, 6,437 graph edges, 1,061 reachable nodes in the largest connected component, a Dijkstra path cost of approximately 4.72, and an A* path cost of approximately 4.68. These figures provided the project with a first quantitative language for discussing the computational consequences of simplifying space.

By the end of the day, the project had a stronger experimental identity, a clearer implementation sequence, and a more defensible research question centered not simply on spatial capture, but on how abstraction choices govern algorithmic behavior in spatial computing systems.

Output: consolidated the week's work into a unified research pipeline with explicit target metrics: 681,270 raw points, 9,842 downsampled points, 1,200 graph nodes, 6,437 graph edges, 1,061 reachable nodes, Dijkstra cost ≈ 4.72, and A cost ≈ 4.68*.

March 21, 2026

Shifted the project emphasis from offline scan analysis toward live spatial acquisition and model maintenance within a running visionOS application. Rather than treating scanning as a one-time capture event followed by later analysis, I began investigating how an Apple Vision Pro app could remain spatially aware while active, continuously updating its internal environmental model as the user moved through space. This marked an important architectural transition in the project: the goal was no longer just to load a representation of space, but to study how a representation can be built, revised, and queried dynamically inside an immersive system.

I used this stage to clarify the distinction between a static point cloud dataset and a live spatial-computing pipeline. A static scan is useful for early abstraction experiments, but a live immersive system introduces additional research questions: how frequently should

environmental updates be incorporated, what counts as a stable enough model for graph construction, and when should an algorithm re-plan rather than preserve its previous internal state? These are important because in a live spatial system, the representation is not fixed. It is provisional and constantly negotiated between sensing, abstraction, and decision-making.

Output: reframed the project from offline spatial analysis toward a live-update architecture in which an immersive Vision Pro app incrementally constructs and maintains an internal model of the surrounding environment for later algorithmic use.

March 22, 2026

Began investigating how ARKit scene reconstruction on Vision Pro could support a continuously updated world model rather than a single exportable mesh. In Xcode, I continued experimenting with the visionOS app structure and the immersive-space pipeline, focusing on how ARKitSession and SceneReconstructionProvider would fit into a longer-lived runtime loop rather than a simple startup check. This was a meaningful design step because it shifted the role of scene reconstruction from “is this supported?” to “how should this stream of geometric updates be incorporated into a stable computational representation?”

At a conceptual level, I treated the incoming scene reconstruction data as a sequence of partial observations rather than a final environment description. That led me to start thinking in terms of incremental representation building: mesh fragments or anchor updates should eventually be merged into a higher-level abstraction that algorithms can use without needing to reason over every incoming geometric revision. In other words, the research focus began moving toward how to preserve algorithmic stability in the presence of perceptual change.

Output: developed a stronger architectural understanding of Vision Pro scene reconstruction as a live stream of geometric updates to be integrated into a persistent internal model rather than treated as a one-time scan result.

March 23, 2026

Focused on the problem of representation persistence in a live immersive application. A central issue I began researching was how a continuously sensing Vision Pro app could maintain a model of the environment that is stable enough for algorithmic reasoning while still responsive to new geometric information. In practice, this means the project cannot rely on raw mesh updates alone. It needs an intermediate layer that stores spatial state in a form that can survive local geometric changes without forcing the entire computational model to be rebuilt from scratch.

This led me to think more seriously about layered spatial representations. One possibility is to maintain a lower-level geometric layer derived from scene reconstruction and a higher-level decision layer, such as a graph or occupancy structure, that is updated selectively rather than regenerated globally. This matters because a system that rebuilds everything on every spatial update may be computationally wasteful and unstable, whereas a system that never revises its abstraction risks making outdated decisions. The research question here is not merely how to scan space, but how to preserve decision-relevant structure while the sensed environment remains in flux.

Output: refined the project toward a multi-layer representation strategy in which live geometric input would feed a more stable higher-level structure for pathfinding and decision-making.

March 24, 2026

Continued exploring the implementation side in Xcode by thinking through how the immersive app should support active modeling while in use, rather than only front-loading all spatial understanding at startup. I treated the immersive application less as a viewer and more as an experimental platform for real-time spatial reasoning. This required clarifying where spatial state should live inside the app, how updates should propagate through that state, and how algorithmic queries could be made against a model that is actively changing while the user remains immersed.

This was also the point where the project began to feel more distinctly like spatial computing research rather than geometry preprocessing. I started treating the app itself as part of the experiment: the sequencing of sensing, abstraction, and decision-making has to be designed carefully because it determines when the system considers its knowledge of the environment sufficient for a meaningful pathfinding or reachability computation. That framing strengthened the overall research direction by tying implementation choices directly to theoretical questions about model validity and algorithmic confidence.

Output: advanced the app design from a simple immersive scaffold toward a research-oriented runtime structure capable of supporting active model construction and later live algorithmic queries.

March 25, 2026

Researched how algorithmic decisions should behave in a partially known space, which became increasingly important once the project shifted toward active Vision Pro-based spatial modeling. In the earlier offline point-cloud case, the graph was generated from a fixed dataset. In the live case, however, the graph or other abstraction may emerge only gradually as the user scans the

space. This raises a fundamentally different decision problem: how should a pathfinding method behave when the environment is incomplete, uncertain, or still being discovered?

I used this stage to sharpen the project's algorithmic direction. Rather than assuming that BFS, Dijkstra, or A* will always operate over a finished map, I began thinking of them as reasoning methods over evolving internal models. This makes the abstraction problem richer, because the system is no longer just optimizing over space; it is optimizing over its current belief about space. That distinction makes the project more interesting and also more aligned with real spatial-computing conditions, where sensed environments are rarely available in perfect final form from the start.

Output: extended the project's algorithmic framing from search over static graphs to decision-making over incrementally constructed and potentially incomplete spatial models.

March 26, 2026

Clarified the relationship between live scene reconstruction, model support, and algorithmic interpretability. By this point, I was no longer thinking of Vision Pro simply as a source of geometry, but as a platform where sensing and reasoning could coexist in real time. This required a stronger conceptual account of what the internal model is supposed to support. The model should not just exist for visualization. It should support queries such as reachability, path plausibility, environmental continuity, and local re-routing as new information arrives.

That led me to refine the project around a more precise systems question: what properties must an internal model have in order to support meaningful algorithmic decisions in an immersive application? Stability, update efficiency, and interpretability emerged as the most important criteria. A model that is too raw may preserve detail but be difficult to reason over. A model that is too simplified may support fast decisions but erase spatial constraints that matter. This balance between fidelity and tractability became a central design theme.

Output: formalized the emerging requirement that the Vision Pro-side representation must support not only spatial capture, but stable, interpretable, and incrementally updateable algorithmic reasoning.

March 27, 2026

Used this period to connect the implementation research more explicitly to the project's core intellectual contribution. The work was no longer only about comparing point clouds, meshes, and graphs as static representational forms. It had become increasingly about understanding how a spatial-computing system maintains a usable internal model while the world is still being

sensed. This is a stronger and more interesting framing because it treats representation not as a static artifact, but as an active computational process.

I also clarified that this live-modeling perspective is exactly where algorithmic decision-making becomes meaningful. A decision is not simply the output of a search procedure. It is the output of a search procedure over a model whose structure depends on how the system has chosen to interpret, simplify, and maintain its knowledge of the environment. This perspective gave the project a more advanced research identity and made the Vision Pro component feel methodologically essential rather than just technically interesting.

Output: elevated the project from a static representation comparison into a study of how live spatial systems construct and maintain the internal models that algorithms ultimately reason over.

March 28, 2026

Refined the implementation roadmap around the idea of incremental model support. Rather than imagining a fully finished map followed by search, I began treating the model as something that should become progressively more useful as scanning proceeds. This implies that the system should eventually support intermediate computational states: partial connectivity, provisional pathways, or local navigable regions that become more globally coherent over time. That is a much more realistic representation of how an immersive spatial system would behave in practice.

I also used this stage to think through how different abstraction choices would affect re-planning behavior. If a graph is updated node-by-node as spatial information arrives, then A* or Dijkstra may need to re-evaluate prior assumptions about the best path. If the abstraction is kept at a coarser level, the system may produce more stable but less precise decisions. This made the project's design space much richer, because the choice of representation now shapes not only one decision, but the frequency and nature of future decisions as the model evolves.

Output: developed a more sophisticated roadmap for incremental spatial model support, including the idea of provisional representations that can support early decisions and later re-planning.

March 29, 2026

Continued grounding the Vision Pro work in actual implementation research by using Xcode as a place to think through how sensing, state, and algorithmic evaluation would coexist in the same immersive application. I treated the existing immersive scaffold not as wasted prototype work, but as the first systems layer necessary for later real-time modeling. The important question during this stage was how to move from "the app can open an immersive space and receive

scene-related input" to "the app can maintain a structured internal state that algorithms can query repeatedly while spatial updates continue in the background."

This strengthened the project considerably, because it made the app architecture itself part of the research methodology. The problem is not just to design an algorithm, but to design a runtime environment in which that algorithm has access to a continually revised model of the world. Thinking through that architecture in Xcode was productive because it clarified the eventual need for state management, abstraction layering, and rules governing when model updates should trigger algorithmic revision.

Output: used the existing immersive Xcode prototype as the basis for a more serious runtime architecture centered on persistent spatial state and repeated algorithmic queries during active scene reconstruction.

THIS IS WHAT I WILL WORK ON AFTER THIS WEEK, SADLY HAVE AN EXAM
The next step is to move from architectural planning to implementation by adding a persistent spatial-state manager inside the visionOS app, storing incoming reconstruction data over time, converting that data into a simplified higher-level structure such as a graph or occupancy-based representation, and then testing when and how pathfinding algorithms should be recomputed as the model changes.

March 30, 2026

Began translating the end-of-week architectural conclusions into a more concrete implementation plan. Rather than continuing to treat live scene reconstruction as a stream of isolated updates, I formalized the next phase around building a persistent spatial-state manager inside the visionOS app. The goal of this layer is to accumulate reconstruction data over time, preserve it in a stable internal structure, and make it queryable by later modeling and pathfinding components. This was an important step because it moved the project from conceptual discussion into a systems-level design problem centered on how a spatial-computing application should remember what it has seen.
Output: defined the next implementation target as a persistent in-app state layer for live reconstruction data.

March 31, 2026

Focused on the relationship between live reconstruction updates and higher-level abstraction stability. I clarified that the system should not immediately expose raw geometric updates to graph algorithms, because doing so would make the reasoning layer too volatile. Instead, I began

outlining a staged internal pipeline in which reconstruction output is first aggregated and normalized, and only then promoted into a more stable representation suitable for decision-making. This sharpened the architecture by distinguishing sensing, state maintenance, and reasoning as separate computational responsibilities.
Output: specified a staged pipeline of reconstruction input → persistent state → stabilized abstraction → algorithmic query.

April 1, 2026

Used this period to think through how the persistent model should be represented internally inside the app. I explored the tradeoffs between maintaining a geometry-heavy structure that preserves fidelity and maintaining a more abstract structure that supports faster algorithmic reasoning. This led me to formalize the idea that the project may require multiple coexisting representations rather than a single universal model: one closer to the incoming scene geometry, and another simplified enough to support graph construction and search.
Output: refined the project toward a layered representation strategy rather than a single all-purpose spatial model.

April 2, 2026

Returned to Xcode and continued reasoning about where this persistent spatial state should live within the runtime structure of the immersive app. I treated the app less as a front-end shell and more as the experimental system itself, which meant thinking more seriously about state ownership, update propagation, and how frequently model changes should become visible to the reasoning layer. This helped clarify that the project is as much about runtime design as it is about geometry or algorithms.
Output: advanced the internal app architecture by treating spatial state management as a core research component rather than a secondary implementation detail.

April 3, 2026

Spent time thinking through how graph updates should occur once live scene data is being accumulated over time. I distinguished between two possible strategies: full graph recomputation after meaningful model changes, or selective/local graph revision as new spatial information is integrated. This was a valuable design step because it framed graph construction not as a one-time preprocessing action, but as an evolving process that must balance stability against responsiveness.
Output: defined graph maintenance itself as a dynamic systems problem, not merely a preprocessing step.

April 4, 2026

Shifted attention toward the problem of decision timing. In a live system, the main issue is not only how to compute a path, but when the system should consider its model complete enough to justify doing so. I began framing this as a confidence and sufficiency problem: the internal model may be partially constructed, yet still usable for local reasoning. This significantly improved the project's research framing because it introduced the idea that spatial-computing decisions are conditioned not just on geometry, but on the evolving adequacy of the internal world model.
Output: clarified that model sufficiency and timing of re-planning are central research questions in the live Vision Pro setting.

April 5, 2026

Read more literature and technical documentation related to incremental mapping, planning in partially observed environments, and real-time model updating. This reading helped place the project in a broader context and reinforced that the key challenge is not simply generating a map, but maintaining a representation that is both computationally tractable and decision-relevant as it changes over time. I used this reading period to strengthen the theoretical motivation behind the persistent-state and layered-abstraction approach.
Output: expanded the literature basis for the project around dynamic modeling, partial observability, and incremental decision-making.

April 6, 2026

Used the literature review to sharpen the project's conceptual distinction between perception, modeling, and reasoning. Rather than collapsing all three into "spatial understanding," I began treating them as separate but interacting layers. This made the project's architecture much more disciplined: perception provides raw updates, modeling converts them into structured state, and reasoning operates over whichever abstraction best preserves decision-relevant information.
Output: formalized a three-layer conceptual framework of perception, modeling, and reasoning for the live spatial pipeline.

April 7, 2026

Revisited the role of algorithms in this evolving architecture. Instead of assuming BFS, Dijkstra, and A* would simply run on a finished graph, I began positioning them as methods that may need to operate over temporally changing models. This broadened the significance of the algorithmic component, because the problem became not just shortest-path search, but search under evolving representational conditions.

Output: extended the project's algorithmic framing from static graph search to reasoning over dynamically revised spatial models.

April 8, 2026

Explored how a coarser intermediate representation might support earlier and more stable decision-making than a finer, rapidly fluctuating graph. This was an important representational insight: higher fidelity is not always better if it undermines stability. I started considering whether an occupancy-style or region-based abstraction might serve as a bridge between live reconstruction output and later graph generation.
Output: refined the abstraction strategy by considering coarser intermediate models that may improve stability in live decision-making.

April 9, 2026

Continued experimenting conceptually in Xcode while thinking through how the immersive application should expose spatial state to later algorithms. I treated the existing prototype as a systems foundation rather than a throwaway mockup, and used it to reason about how live updates, internal storage, and algorithmic queries might coexist within one application lifecycle. This made the project feel more cohesive, because the software architecture and research question were beginning to align much more clearly.
Output: integrated the immersive app prototype more directly into the project's methodological structure.

April 10, 2026

Spent time consolidating notes and clarifying the project's next engineering milestone: implementing a component that stores reconstruction-derived state over time and supports later conversion into a decision-level structure. This was less about visible feature output and more about making the next coding phase coherent before adding more moving pieces. I used this day to reduce ambiguity in the implementation order so that future work would be more targeted and less fragmented.
Output: clarified the immediate engineering milestone as persistent storage and organization of reconstruction-derived spatial state.

April 11, 2026

Read more literature and documentation related to scene reconstruction, runtime mapping, and navigation over evolving maps. This reading period also helped me refine how to talk about the

project academically: the work is not about “making a scan,” but about understanding how a spatial-computing system builds internal structure that becomes sufficient for reasoning. I used this to improve the theoretical positioning of the project for later writeups and discussions.
Output: strengthened the project’s academic framing through targeted reading on dynamic mapping and navigation.

April 12, 2026

A lighter work day due to other academic obligations, but I still used some time to review the overall architecture and think through how the live Vision Pro side and the offline point-cloud side should complement rather than duplicate one another. This helped reinforce that the offline datasets are useful for early abstraction experiments, while the Vision Pro side is necessary for studying model construction and algorithmic behavior in real time.
Output: clarified the division of labor between offline dataset experiments and live immersive validation.

April 13, 2026

Returned to the project by refining the idea of model persistence under local change. I focused on the question of how much of the internal abstraction should be preserved when the sensed environment updates. This led me to think more carefully about spatial memory, revision boundaries, and what kinds of updates should trigger full re-evaluation versus local adjustment.
Output: expanded the architectural design around partial revision and preservation of previously constructed spatial state.

April 14, 2026

Spent more time in planning and reading than in direct coding, but this was still productive because it clarified the project’s central systems tension: the representation must remain flexible enough to absorb new spatial evidence, but stable enough that algorithmic outputs remain interpretable. This balance between adaptability and stability increasingly became the central design criterion for the project.
Output: identified stability-versus-adaptability as a core evaluation lens for the live model architecture.

April 15, 2026

Focused on how algorithmic re-computation should be governed in the final system. I began framing re-planning as an event-driven process rather than something that should happen

continuously at every minor geometric update. This matters because constant re-planning could make the reasoning layer unstable and computationally wasteful, whereas delayed re-planning could make the system rely on outdated assumptions.
Output: introduced the idea of trigger-based algorithmic re-computation tied to meaningful model changes rather than raw update frequency.

April 16, 2026

Used the day to refine the project's language around decision-relevant structure. I clarified that the ultimate purpose of the internal model is not to preserve every geometric detail, but to preserve the aspects of space that matter for reachability, path plausibility, local continuity, and route revision. This made the project's representational goals much sharper and more defensible.
Output: narrowed the model's intended purpose to preserving decision-relevant spatial structure rather than full geometric fidelity.

April 17, 2026

Another somewhat lighter day because of competing work, but I still reviewed notes, documentation, and prior logs so the project would remain coherent. I used the time to tighten the transition between the previously completed point-cloud/graph experiments and the newer live-modeling direction. This prevented the project from splitting into two disconnected halves.
Output: maintained continuity between the offline abstraction experiments and the live Vision Pro modeling architecture.

April 18, 2026

Returned to the implementation roadmap and clarified the internal ordering of components: reconstruction data first needs to be retained, then stabilized, then abstracted, and only afterward passed to graph or pathfinding logic. This sequencing helped eliminate earlier ambiguity about whether graph construction should happen immediately or only after enough state has accumulated.
Output: finalized a clearer component sequence of retention → stabilization → abstraction → algorithmic reasoning.

April 19, 2026

Read additional literature on navigation under uncertainty and incremental environment modeling, using it mainly to pressure-test the project's current direction. This reading reinforced that my current architectural emphasis is justified: in live spatial systems, model maintenance is

not a peripheral concern but one of the main determinants of decision quality.
Output: reinforced the theoretical legitimacy of focusing on persistent model maintenance as central to algorithmic performance.

April 20, 2026

Used this period to think more concretely about how the higher-level abstraction should eventually be queried. I refined the project's notion of output beyond just "a path." The relevant outputs in the live setting should also include reachability updates, local path plausibility, and cases where the system chooses not to commit to a path because the model remains insufficiently stable. This broadened the project's decision framework in a useful way.
Output: expanded the set of meaningful live-system outputs to include uncertainty-aware and model-conditioned decision behaviors.

April 21, 2026

A lighter day because of other obligations, but I still used project time to organize notes and align the literature findings with the implementation roadmap. This was helpful because the project now has enough conceptual depth that keeping the written research framing synchronized with the software design is itself important work.
Output: consolidated literature insights and architectural notes into a more unified research direction.

April 22, 2026

Prepared for the next active coding phase by consolidating the project's immediate objective into one clear implementation target: add a persistent spatial-state manager inside the visionOS app that stores incoming reconstruction information over time and serves as the source for later graph or occupancy-style abstraction. This sharpened the project considerably because it converted a br

Used this period to think more concretely about how the higher-level abstraction should eventually be queried. I refined the project's notion of output beyond just "a path." The relevant outputs in the live setting should also include reachability updates, local path plausibility, and cases where the system chooses not to commit to a path because the model remains insufficiently stable. This broadened the project's decision framework in a useful way.
Output: expanded the set of meaningful live-system outputs to include uncertainty-aware and model-conditioned decision behaviors.

April 21, 2026

A lighter day because of other obligations, but I still used project time to organize notes and align the literature findings with the implementation roadmap. This was helpful because the project now has enough conceptual depth that keeping the written research framing synchronized with the software design is itself important work.
Output: consolidated literature insights and architectural notes into a more unified research direction.

April 22, 2026

Prepared for the next active coding phase by consolidating the project's immediate objective into one clear implementation target: add a persistent spatial-state manager inside the visionOS app that stores incoming reconstruction information over time and serves as the source for later graph or occupancy-style abstraction. This sharpened the project considerably because it converted a broad architectural direction into a specific next build step.
Output: finalized the next implementation milestone as persistent reconstruction-state storage inside the immersive Vision Pro application.

April 23, 2026

Used the day to synthesize the recent work into a more mature research trajectory. Over this stretch, progress was less about producing flashy outputs and more about making sure the project does not become shallow or mechanically repetitive. I have been actively working through how an Apple Vision Pro application can scan while active, support internal model construction over time, and enable algorithmic decisions that are grounded in a representation stable enough to be meaningful but flexible enough to keep updating. I also continued reading literature and documentation on incremental reconstruction, dynamic mapping, and planning under partial observability, which helped validate that the current direction is not wasted time but exactly the kind of systems-level problem that makes the project stronger. There were a few lighter days due to exams and other coursework, but overall the period was productive in moving the project from "interesting prototype" toward a more defensible live spatial-computing research system.
Output: consolidated the recent phase around a clear live-systems research question: how to support persistent internal world modeling and decision-relevant abstraction in an immersive Vision Pro application while reconstruction is still ongoing.

April 25, 2026
Focused on translating the persistent spatial state manager idea into a clearer software design. I outlined what information the Vision Pro app would need to preserve over time, including reconstruction derived geometry, update timestamps, and simplified spatial regions that could later be queried by pathfinding algorithms. This helped me separate raw scene input from the higher level model that the algorithms should actually reason over. Instead of treating every

incoming update as equally important, I started thinking about how the app could filter and retain only decision relevant spatial changes.

Output: drafted a clearer design for the persistent spatial-state manager, including what data should be stored, how updates should be organized, and how this layer would support later graph or occupancy-based abstraction.

April 27, 2026
Worked on strengthening the connection between the live Vision Pro system and the offline point cloud experiments. I reviewed how the earlier Scaniverse point cloud pipeline converted raw spatial data into a graph, then compared that process to what would need to happen in a live immersive app. The biggest difference is that the offline graph is built from a finished dataset, while the live system would need to build and revise its internal model gradually. This helped clarify that the offline work is still useful as a baseline, but the live system introduces new questions about timing, stability, and when algorithms should recompute.

Output: clarified how the offline point-cloud workflow can serve as a controlled comparison for the live Vision Pro pipeline, especially around graph construction, model stability, and algorithmic re planning.

April 30, 2026
Used this period to consolidate the work from weeks 9 and 10 into a clearer next step plan. The main progress during this stretch was not a flashy new visual output, but a stronger research architecture for how live scene reconstruction should support algorithmic reasoning. I refined the project's central idea around persistent world modeling: the system needs to remember what it has sensed, stabilize that information into a usable abstraction, and then allow algorithms like BFS, Dijkstra, or A* to query the model without being disrupted by every small geometry update. This made the next coding milestone much more concrete and gave the project a stronger systems level direction.

Output: finalized the immediate next step as implementing the persistent spatial-state layer in the visionOS app, then connecting that layer to a simplified decision-level representation for future pathfinding and reachability experiments**.**